🎮 Documentación Técnica

Juego del Gato (Tres en Raya) - Sistema de IA y Gestión de Partidas

Universidad Veracruzana | Programación

📚 Introducción

Descripción del Proyecto

Este proyecto implementa el clásico juego del Gato (también conocido como Tres en Raya o Tic-Tac-Toe) en C++. El sistema incluye múltiples modos de juego, inteligencia artificial con dos niveles de dificultad, sistema de estadísticas y una interfaz de usuario en consola con diseño visual atractivo.

Características Principales

- **Modo Jugador vs Jugador:** Permite que dos personas jueguen entre sí
- **Modo Jugador vs IA:** Incluye dos niveles de dificultad (Fácil y Difícil)
- **Sistema de Estadísticas:** Rastrea victorias, derrotas, empates y porcentajes
- **Validación de Entrada:** Verifica que los movimientos sean válidos
- **Interfaz Visual:** Utiliza caracteres Unicode y formato de cajas para mejorar la experiencia

Tecnologías Utilizadas

Tecnología	Versión	Propósito
C++	C++11	Lenguaje de programación principal
STL (Standard Template Library)	-	Manejo de strings, entrada/salida
chrono & thread	C++11	Temporizadores y pausas visuales
ANSI Escape Codes	-	Posicionamiento del cursor en terminal

🏗️ Estructuras de Datos

Estructura Jugador

`struct Jugador`

Representa a un jugador en el juego, ya sea humano o controlado por inteligencia artificial.

```
struct Jugador {
    string nombre;      // Nombre del jugador
    char simbolo;       // 'X' o 'O'
    bool esHumano;      // true si es humano, false si es IA
};
```

Campo	Tipo	Descripción
`nombre`	string	Almacena el nombre del jugador o "IA Fácil"/"IA Difícil"
`simbolo`	char	Carácter que representa al jugador en el tablero ('X' o 'O')
`esHumano`	bool	Indica si el jugador es controlado por humano o por IA

Estructura Estadisticas

`struct Estadisticas`

Mantiene un registro de los resultados de las partidas jugadas durante la sesión.

```
struct Estadisticas {
    int victorias;       // Victorias del jugador 1
    int derrotas;        // Derrotas del jugador 1
    int empates;         // Partidas empatadas
    int partidasJugadas; // Total de partidas
};
```

💡 Nota Importante

Las estadísticas se reinician cada vez que se cierra el programa. Para mantener un historial permanente, sería necesario implementar persistencia en archivos.

⚙️ Funciones Principales

1. Funciones del Tablero

inicializarTablero()

`void inicializarTablero(char tablero[3][3])`

Propósito: Inicializa todas las posiciones del tablero con espacios en blanco, preparándolo para una nueva partida.

```
void inicializarTablero(char tablero[3][3]) {
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            tablero[i][j] = ' ';
        }
    }
}
```

Funcionamiento:

1. Recorre cada fila del tablero (i de 0 a 2)
2. Para cada fila, recorre cada columna (j de 0 a 2)
3. Asigna un espacio en blanco ' ' a cada posición

mostrarTablero()

`void mostrarTablero(char tablero[3][3])`

Propósito: Muestra el tablero en la consola con un formato visual atractivo.

```
void mostrarTablero(char tablero[3][3]) {
    cout << "\n";
    for(int i = 0; i < 3; i++) {
        cout << "  ";
        for(int j = 0; j < 3; j++) {
            cout << " " << tablero[i][j] << " ";
            if(j < 2) cout << "|";
        }
        cout << endl;
        if(i < 2) cout << "  ---|---|---" << endl;
    }
    cout << "\n";
}
```

Ejemplo de salida:

```
 X | O | X
---|---|---
 O | X |
---|---|---
   |   | O
```

tableroLleno()

`bool tableroLleno(char tablero[3][3])`

Propósito: Verifica si el tablero está completamente lleno (condición de empate).

```
bool tableroLleno(char tablero[3][3]) {
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            if(tablero[i][j] == ' ') {
                return false;  // Encontró un espacio vacío
            }
        }
    }
    return true;  // No hay espacios vacíos
}
```

Retorna: `true` si todas las casillas están ocupadas, `false` si aún hay espacios disponibles.

2. Funciones de Verificación de Ganador

verificarFilas()

`bool verificarFilas(char tablero[3][3], char simbolo)`

Propósito: Verifica si hay tres símbolos iguales en alguna fila horizontal.

```
bool verificarFilas(char tablero[3][3], char simbolo) {
    for(int i = 0; i < 3; i++) {
        if(tablero[i][0] == simbolo &&
           tablero[i][1] == simbolo &&
           tablero[i][2] == simbolo) {
            return true;
        }
    }
    return false;
}
```

Lógica: Recorre cada fila y verifica si las tres columnas tienen el mismo símbolo.

verificarColumnas()

`bool verificarColumnas(char tablero[3][3], char simbolo)`

Propósito: Verifica si hay tres símbolos iguales en alguna columna vertical.

```
bool verificarColumnas(char tablero[3][3], char simbolo) {
    for(int j = 0; j < 3; j++) {
        if(tablero[0][j] == simbolo &&
           tablero[1][j] == simbolo &&
           tablero[2][j] == simbolo) {
            return true;
        }
    }
    return false;
}
```

verificarDiagonales()

`bool verificarDiagonales(char tablero[3][3], char simbolo)`

Propósito: Verifica ambas diagonales del tablero.

```
bool verificarDiagonales(char tablero[3][3], char simbolo) {
    // Diagonal principal (↘)
    if(tablero[0][0] == simbolo &&
       tablero[1][1] == simbolo &&
       tablero[2][2] == simbolo) {
        return true;
    }

    // Diagonal secundaria (↙)
    if(tablero[0][2] == simbolo &&
       tablero[1][1] == simbolo &&
       tablero[2][0] == simbolo) {
        return true;
    }

    return false;
}
```

Diagonales del Tablero

Diagonal principal: posiciones [0][0], [1][1], [2][2]

Diagonal secundaria: posiciones [0][2], [1][1], [2][0]

verificarGanador()

`bool verificarGanador(char tablero[3][3], char simbolo)`

Propósito: Función principal que combina todas las verificaciones.

```
bool verificarGanador(char tablero[3][3], char simbolo) {
    return verificarFilas(tablero, simbolo) ||
           verificarColumnas(tablero, simbolo) ||
           verificarDiagonales(tablero, simbolo);
}
```

Esta función utiliza el operador lógico OR (`||`) para retornar `true` si cualquiera de las verificaciones es exitosa.

3. Funciones de Validación

validarEntradaNumerica()

`bool validarEntradaNumerica(int& valor)`

Propósito: Verifica que la entrada del usuario sea un número válido.

```
bool validarEntradaNumerica(int& valor) {
    if(cin.fail()) {
        cin.clear();                    // Limpia el estado de error
        cin.ignore(10000, '\n');        // Descarta entrada inválida
        cout << "\n❌ Error: Debes ingresar un numero." << endl;
        return false;
    }
    return true;
}
```

⚠️ Manejo de Errores

Si el usuario ingresa texto en lugar de números, `cin.fail()` retorna `true`. Es necesario limpiar el buffer y el estado de error antes de continuar.

validarRango()

`bool validarRango(int valor, int min, int max)`

Propósito: Verifica que un número esté dentro del rango permitido (1-3).

```
bool validarRango(int valor, int min, int max) {
    if(valor < min || valor > max) {
        cout << "\n❌ Error: Debe estar entre " << min
             << " y " << max << "." << endl;
        return false;
    }
    return true;
}
```

movimientoValido()

`bool movimientoValido(char tablero[3][3], int fila, int columna)`

Propósito: Verifica que una casilla esté dentro del tablero y disponible.

```
bool movimientoValido(char tablero[3][3], int fila, int columna) {
    // Verificar límites del tablero
    if(fila < 0 || fila > 2 || columna < 0 || columna > 2) {
        cout << "\n❌ Posicion fuera del tablero." << endl;
        return false;
    }

    // Verificar que la casilla esté vacía
    if(tablero[fila][columna] != ' ') {
        cout << "\n❌ Esa casilla ya esta ocupada." << endl;
        return false;
    }

    return true;
}
```

🤖 Sistema de Inteligencia Artificial

Descripción General

El sistema de IA implementa dos niveles de dificultad: **Fácil** (movimientos aleatorios) y **Difícil** (estrategia avanzada con priorización de movimientos).

IA Nivel Fácil

movimientoIAFacil()

`void movimientoIAFacil(char tablero[3][3], char simbolo)`

Estrategia: Selecciona casillas vacías aleatoriamente sin análisis estratégico.

```
void movimientoIAFacil(char tablero[3][3], char simbolo) {
    int fila, columna;

    // Buscar casilla vacía aleatoria
    do {
        fila = rand() % 3;      // Número aleatorio 0-2
        columna = rand() % 3;   // Número aleatorio 0-2
    } while(tablero[fila][columna] != ' ');

    tablero[fila][columna] = simbolo;
    cout << "\n🤖 IA juega en: (" << (fila + 1)
         << ", " << (columna + 1) << ")" << endl;
}
```

Características:

- No analiza el estado del juego
- No busca ganar ni bloquear al oponente
- Ideal para jugadores principiantes
- Tiempo de respuesta inmediato

IA Nivel Difícil

La IA difícil implementa una jerarquía de estrategias ordenadas por prioridad:

🎯 Jerarquía de Decisiones

1. **Ganar:** Si puede ganar en el siguiente movimiento, lo hace
2. **Bloquear:** Si el oponente puede ganar, lo bloquea
3. **Centro:** Si está libre, toma el centro (posición estratégica)
4. **Esquinas:** Toma una esquina disponible
5. **Aleatorio:** Como último recurso, movimiento aleatorio

intentarGanar()

`bool intentarGanar(char tablero[3][3], char simbolo)`

Propósito: Busca un movimiento que resulte en victoria inmediata.

```
bool intentarGanar(char tablero[3][3], char simbolo) {
    // Probar cada casilla vacía
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            if(tablero[i][j] == ' ') {
                // Simular movimiento
                tablero[i][j] = simbolo;

                // ¿Este movimiento gana?
                if(verificarGanador(tablero, simbolo)) {
                    cout << "\n🧠 IA juega en: (" << (i + 1)
                         << ", " << (j + 1) << ")" << endl;
                    return true;  // Mantener el movimiento
                }

                // Deshacer simulación
                tablero[i][j] = ' ';
            }
        }
    }
    return false;  // No hay movimiento ganador
}
```

Algoritmo:

1. Recorre todas las casillas del tablero
2. Para cada casilla vacía, simula colocar el símbolo de la IA
3. Verifica si ese movimiento resulta en victoria
4. Si gana, mantiene el movimiento y retorna `true`
5. Si no gana, deshace la simulación y continúa buscando

bloquearOponente()

`bool bloquearOponente(char tablero[3][3], char simboloIA, char simboloOponente)`

Propósito: Detecta si el oponente puede ganar en su próximo turno y lo bloquea.

```
bool bloquearOponente(char tablero[3][3], char simboloIA, char simboloOponente) {
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            if(tablero[i][j] == ' ') {
                // Simular movimiento del oponente
                tablero[i][j] = simboloOponente;

                // ¿El oponente ganaría?
                if(verificarGanador(tablero, simboloOponente)) {
                    // Bloquear con nuestro símbolo
                    tablero[i][j] = simboloIA;
                    cout << "\n🛡️  IA bloquea en: (" << (i + 1)
                         << ", " << (j + 1) << ")" << endl;
                    return true;
                }

                // Deshacer simulación
                tablero[i][j] = ' ';
            }
        }
    }
    return false;  // No hay amenaza inmediata
}
```

💡 Concepto Clave: Simulación de Movimientos

Tanto `intentarGanar()` como `bloquearOponente()` utilizan la técnica de **simulación**: colocan temporalmente un símbolo, verifican el resultado, y deshacen el cambio si no es favorable. Esta técnica es fundamental en algoritmos de juegos.

movimientoIADificil()

`void movimientoIADificil(char tablero[3][3], char simboloIA, char simboloOponente)`

Propósito: Función principal que coordina toda la estrategia de la IA difícil.

```
void movimientoIADificil(char tablero[3][3], char simboloIA, char simboloOponente) {
    // 1. PRIORIDAD MÁXIMA: Ganar
    if(intentarGanar(tablero, simboloIA)) return;

    // 2. Bloquear al oponente
    if(bloquearOponente(tablero, simboloIA, simboloOponente)) return;

    // 3. Tomar el centro si está disponible
    if(tablero[1][1] == ' ') {
        tablero[1][1] = simboloIA;
        cout << "\n🎯 IA toma el centro: (2, 2)" << endl;
        return;
    }

    // 4. Tomar una esquina
    int esquinas[4][2] = {{0,0}, {0,2}, {2,0}, {2,2}};
    for(int i = 0; i < 4; i++) {
        int fila = esquinas[i][0];
        int col = esquinas[i][1];
        if(tablero[fila][col] == ' ') {
            tablero[fila][col] = simboloIA;
            cout << "\n📐 IA toma esquina: (" << (fila + 1)
                 << ", " << (col + 1) << ")" << endl;
            return;
        }
    }


// 5. ÚLTIMO RECURSO: Movimiento aleatorio
movimientoIAFacil(tablero, simboloIA);

}
```

Prioridad	Estrategia	Razón
①	Ganar	Victoria inmediata tiene máxima prioridad
②	Bloquear	Prevenir derrota del oponente
③	Centro	Posición estratégica central
④	Esquinas	Más oportunidades de formar líneas
5	Aleatorio	Cuando no hay mejor opción

Comparación de Niveles de IA

Característica	IA Fácil	IA Difícil
Análisis del tablero	❌ No	✅ Completo
Busca ganar	❌ No	✅ Sí (prioridad 1)
Bloquea oponente	❌ No	✅ Sí (prioridad 2)
Estrategia posicional	❌ Ninguna	✅ Centro y esquinas
Nivel de dificultad	🟢 Principiante	🔴 Avanzado

🎮 Lógica del Juego

jugarPartida()

`void jugarPartida(char tablero[3][3], Jugador& j1, Jugador& j2, Estadisticas& stats)`

Propósito: Controla el flujo completo de una partida desde el inicio hasta su conclusión.

```
void jugarPartida(char tablero[3][3], Jugador& j1, Jugador& j2, Estadisticas& stats) {
inicializarTablero(tablero);
bool juegoTerminado = false;
int turnoActual = 0;

while(!juegoTerminado) {
    system("clear");
    mostrarTablero(tablero);

    Jugador& actual = (turnoActual == 0) ? j1 : j2;

    // Turno del jugador actual
    if(actual.esHumano) {
        // Lógica para jugador humano
    } else {
        // Lógica para IA
    }

    // Verificar victoria o empate
    if(verificarGanador(tablero, actual.simbolo)) {
        // Actualizar estadísticas
        juegoTerminado = true;
    } else if(tableroLleno(tablero)) {
        // Empate
        juegoTerminado = true;
    }

    turnoActual = 1 - turnoActual;  // Cambiar turno
}

stats.partidasJugadas++;


}
```

🔄 Control de Turnos

La variable `turnoActual` alterna entre 0 y 1 usando la fórmula: `turnoActual = 1 - turnoActual`

- Si turnoActual = 0, entonces: 1 - 0 = 1
- Si turnoActual = 1, entonces: 1 - 1 = 0

Flujo de una Partida

```
          ┌──────────────────────┐
          │  Inicializar tablero vacío  │
          └──────────────────────┘
                     │
                     ▼
         ┌─────────────────────┐
         │   Mostrar tablero actual    │
         └─────────────────────┘
                     │
                     ▼
          ┌──────────────────────┐
          │  ¿Jugador humano o IA?      │
          └──────────────────────┘
                │            │
             Humano          IA
                │            │
                ▼            ▼
          ┌─────────┐  ┌──────────────┐
          │ Solicitar │  │ Calcular      │
          │ entrada   │  │ movimiento    │
          │ fila/col  │  │ estratégico   │
          └─────────┘  └──────────────┘
                │            │
                └─────┬──────┘
                      ▼
              ┌──────────────┐
              │ Colocar símbolo │
              └──────────────┘
                      │
                      ▼
              ┌──────────────┐
              │  ¿Hay ganador?  │
              └──────────────┘
                  │ Sí     │ No
                  │        ▼
                  │    ┌──────────┐
                  │    │ ¿Tablero  │
                  │    │  lleno?   │
                  │    └──────────┘
                  │      │ Sí  │ No
                  ▼      ▼     │
             ┌─────────┐       │
             │ Terminar  │       │
             │  partida  │       │
             └─────────┘       │
                               ▼
                        ┌────────────┐
                        │ Cambiar turno │
                        └────────────┘
                               │
                               └────► (Volver al inicio)
```

modoJugadorVsJugador()

`void modoJugadorVsJugador(Estadisticas& stats)`

Propósito: Configura e inicia una partida entre dos jugadores humanos.

```
void modoJugadorVsJugador(Estadisticas& stats) {
char tablero[3][3];
Jugador j1, j2;

j1.esHumano = true;
j2.esHumano = true;

cout << "Nombre del Jugador 1 (X): ";
cin >> j1.nombre;
cout << "Nombre del Jugador 2 (O): ";
cin >> j2.nombre;

j1.simbolo = 'X';
j2.simbolo = 'O';

jugarPartida(tablero, j1, j2, stats);
}
```

modoJugadorVsIA()

`void modoJugadorVsIA(Estadisticas& stats, int dificultad)`

Propósito: Configura e inicia una partida contra la IA.

```
void modoJugadorVsIA(Estadisticas& stats, int dificultad) {
char tablero[3][3];
Jugador j1, j2;

dificultadGlobal = dificultad;

j1.esHumano = true;
j2.esHumano = false;  // IA

cout << "Tu nombre: ";
cin >> j1.nombre;

j2.nombre = (dificultad == 1) ? "IA Facil" : "IA Dificil";

j1.simbolo = 'X';
j2.simbolo = 'O';

jugarPartida(tablero, j1, j2, stats);

}
```

⚙️ Variable Global dificultadGlobal

Se utiliza una variable global para comunicar el nivel de dificultad a la función `jugarPartida()`. Valores posibles:

- `1` = IA Fácil (movimientos aleatorios)
- `2` = IA Difícil (estrategia avanzada)

📊 Sistema de Estadísticas

mostrarEstadisticas()

`void mostrarEstadisticas(Estadisticas stats)`

Propósito: Muestra un resumen completo de las partidas jugadas durante la sesión.

```
void mostrarEstadisticas(Estadisticas stats) {
system("clear");

cout << "\n╔══════════════════════════════╗" << endl;
cout << "║         📊 ESTADISTICAS              ║" << endl;
cout << "╚══════════════════════════════╝" << endl;

cout << "\n📈 Partidas jugadas: " << stats.partidasJugadas << endl;
cout << "🏆 Victorias: " << stats.victorias << endl;
cout << "💀 Derrotas: " << stats.derrotas << endl;
cout << "🤝 Empates: " << stats.empates << endl;

if(stats.partidasJugadas > 0) {
    float porcentaje = ((float)stats.victorias / stats.partidasJugadas) * 100;
    cout << "\n✨ Porcentaje de victorias: "
         << fixed << setprecision(1) << porcentaje << "%" << endl;
}


}
```

Cálculo del Porcentaje

```
porcentaje = (victorias / partidasJugadas) × 100
```

Ejemplo: Si un jugador ganó 7 de 10 partidas:

```
porcentaje = (7 / 10) × 100 = 70%
```

reiniciarEstadisticas()

`void reiniciarEstadisticas(Estadisticas& stats)`

Propósito: Permite al usuario reiniciar todas las estadísticas con confirmación.

```
void reiniciarEstadisticas(Estadisticas& stats) {
char confirmar;

cout << "\n⚠️  ¿Estas seguro de reiniciar las estadisticas? (S/N): ";
cin >> confirmar;

if(confirmar == 'S' || confirmar == 's') {
    stats.victorias = 0;
    stats.derrotas = 0;
    stats.empates = 0;
    stats.partidasJugadas = 0;

    cout << "\n✅ Estadisticas reiniciadas con exito." << endl;
} else {
    cout << "\n❌ Operacion cancelada." << endl;
}

}
```

✅ Buena Práctica: Confirmación de Acciones Destructivas

Siempre es recomendable pedir confirmación antes de realizar acciones que no se pueden deshacer, como borrar datos o reiniciar estadísticas.

🔧 Compilación y Ejecución

Requisitos del Sistema

Componente	Requisito	Notas
Compilador	g++ con soporte C++11	GCC 4.8+ o Clang 3.3+
Sistema Operativo	Linux, macOS, Unix	Requiere terminal compatible con ANSI
Bibliotecas	STL estándar de C++	Incluidas con el compilador

Estructura de Archivos

```
proyecto_gato/
      │
      ├── FuncionesJuegoGato.h    ← Archivo de cabecera con todas las funciones
      └── main.cpp                ← Archivo principal con la función main()
```

Contenido de main.cpp

```
#include "FuncionesJuegoGato.h"
                      int main() {
                      menuPrincipal();
                      return 0;
                      }
```

Comandos de Compilación

Compilación Básica

```
g++ -o juego_gato main.cpp -std=c++11
```

Compilación con Warnings (Recomendado)

```
g++ -o juego_gato main.cpp -std=c++11 -Wall -Wextra -pedantic
```

Compilación Optimizada

```
g++ -o juego_gato main.cpp -std=c++11 -O2
```

Flag	Descripción
`-o juego_gato`	Especifica el nombre del archivo ejecutable
`-std=c++11`	Usa el estándar C++11 (requerido para chrono y thread)
`-Wall`	Activa la mayoría de warnings del compilador
`-Wextra`	Activa warnings adicionales no incluidos en -Wall
`-pedantic`	Genera warnings sobre código no estándar
`-O2`	Optimización de nivel 2 (mejora rendimiento)

Ejecución del Programa

En Linux/macOS

```
./juego_gato
```

Dar Permisos de Ejecución (si es necesario)

```
chmod +x juego_gato
                       ./juego_gato
```

⚠️ Nota para Windows

Este programa utiliza comandos específicos de Unix/Linux (`system("clear")` y códigos ANSI). Para ejecutarlo en Windows, necesitarías:

- Reemplazar `system("clear")` por `system("cls")`
- Usar Windows Terminal o una terminal compatible con ANSI
- O ejecutarlo en WSL (Windows Subsystem for Linux)

Solución de Problemas Comunes

Error	Causa	Solución
`g++: command not found`	g++ no está instalado	Instalar: `sudo apt install g++` (Ubuntu) o `xcode-select --install` (macOS)
`error: 'this_thread' is not a member of 'std'`	Falta flag -std=c++11	Agregar `-std=c++11` al comando de compilación
`FuncionesJuegoGato.h: No such file`	Archivo no encontrado	Verificar que el archivo .h esté en el mismo directorio
`Permission denied`	Sin permisos de ejecución	Ejecutar: `chmod +x juego_gato`

🔄 Flujo del Programa Completo

menuPrincipal()

`void menuPrincipal()`

Propósito: Punto de entrada principal que controla todo el ciclo de vida del programa.

```
void menuPrincipal() {
int op;

// Inicializar estadisticas UNA VEZ
Estadisticas stats;
stats.victorias = 0;
stats.derrotas = 0;
stats.empates = 0;
stats.partidasJugadas = 0;

// Inicializar generador aleatorio UNA VEZ
srand(time(0));

do {
    // Mostrar menú
    // Leer opción

    switch(op) {
        case 1: modoJugadorVsJugador(stats); break;
        case 2: modoJugadorVsIA(stats, 1); break;
        case 3: modoJugadorVsIA(stats, 2); break;
        case 4: mostrarInstrucciones(); break;
        case 5: mostrarEstadisticas(stats); break;
        case 6: reiniciarEstadisticas(stats); break;
        case 7: mostrarCreditos(); break;
        case 8: /* Salir */ break;
    }

} while(op != 8);
}
```

📌 Puntos Clave

- Las estadísticas se inicializan **antes** del bucle do-while para que persistan entre partidas
- `srand(time(0))` se llama solo una vez al inicio para evitar patrones repetitivos
- El bucle continúa hasta que el usuario selecciona la opción 8 (Salir)
- Las estadísticas se pasan por referencia (`&stats`) para que se actualicen correctamente

Diagrama de Flujo Completo

```
              INICIO
                │
                ▼
       ┌──────────────────┐
       │  Inicializar stats  │
       │  srand(time(0))     │
       └──────────────────┘
                │
                ▼
       ┌──────────────────┐
       │   Mostrar Menú      │
       │   Principal         │
       └──────────────────┘
                │
                ▼
       ┌──────────────────┐
       │  Leer opción (1-8)  │
       └──────────────────┘
                │
       ┌────────┴────────┐
       │                  │
  1-3: Jugar          4-7: Otras
       │                  │
       ▼                  ▼
 ┌────────────┐   ┌──────────────┐
 │ modoJugador │   │ Instrucciones │
 │ VsJugador   │   │ Estadísticas  │
 │ VsIA        │   │ Reiniciar Stats │
 └────────────┘   │ Créditos      │
       │          └──────────────┘
       ▼                  │
 ┌────────────┐           │
 │ jugarPartida()│         │
 └────────────┘           │
       │                  │
       ▼                  │
 ┌────────────┐           │
 │ Actualizar  │           │
 │ Estadísticas │          │
 └────────────┘           │
       │                  │
       └────────┬─────────┘
                │
                ▼
        ┌────────────┐
        │ ¿Opción = 8? │
        └────────────┘
            │ No    │ Sí
            │       │
            └──┐    ▼
               │  ┌────────────┐
               │  │ Barra de    │
               │  │ Progreso    │
               │  │ Salir       │
               │  └────────────┘
               │        │
               └────────┴──────────► FIN
```

Funciones Auxiliares de Interfaz

gotoxy()

`void gotoxy(int row, int col)`

Propósito: Posiciona el cursor en coordenadas específicas de la terminal usando códigos ANSI.

```
void gotoxy(int row, int col) {
cout << "\x1b[" << row << ";" << col << "H";
}
```

Utiliza la secuencia de escape ANSI `\x1b[row;colH` donde:

- `\x1b` es el carácter ESC (escape)
- `[row;colH` especifica las coordenadas

simularBarraProgreso()

`void simularBarraProgreso(int duracionMs, int longitudBarra)`

Propósito: Muestra una animación de barra de progreso al salir del programa.

```
void simularBarraProgreso(int duracionMs, int longitudBarra) {
const int PASOS_TOTALES = 100;
int tiempoPorPaso = duracionMs / PASOS_TOTALES;

for (int progreso = 0; progreso <= PASOS_TOTALES; ++progreso) {
    double porcentaje = (double)progreso / PASOS_TOTALES;
    int bloquesLlenos = (int)(porcentaje * longitudBarra);
    int bloquesVacios = longitudBarra - bloquesLlenos;
    int porcentajeVisual = (int)(porcentaje * 100);

    gotoxy(19, 3);
    cout << "[";

    // Dibujar bloques llenos
    for (int i = 0; i < bloquesLlenos; ++i) {
        cout << BLOQUE_GRUESO;  // █
    }

    // Dibujar bloques vacíos
    for (int i = 0; i < bloquesVacios; ++i) {
        cout << ESPACIO_VACIO;  //
    }

    cout << "] " << porcentajeVisual << "%" << flush;

    this_thread::sleep_for(chrono::milliseconds(tiempoPorPaso));
}

}
```

Ejemplo visual:

```
[████████████████████            ] 60%

[███████████████████████████     ] 85%

[████████████████████████████████] 100%
```

✨ Mejores Prácticas Implementadas

1. Modularidad

El código está organizado en funciones específicas con responsabilidades únicas:

- Funciones de tablero (inicializar, mostrar, verificar)
- Funciones de validación (entrada, rango, movimiento)
- Funciones de IA (fácil, difícil, estrategias)
- Funciones de estadísticas
- Funciones de menú e interfaz

2. Validación de Entrada

Todas las entradas del usuario son validadas en múltiples niveles:

- **Tipo de dato:** `validarEntradaNumerica()`
- **Rango válido:** `validarRango()`
- **Disponibilidad:** `movimientoValido()`

3. Uso de Estructuras

Las estructuras `Jugador` y `Estadisticas` agrupan datos relacionados, mejorando la legibilidad y mantenimiento del código.

4. Paso por Referencia

```
// Incorrecto - se copia toda la estructura
void funcion(Estadisticas stats);
// Correcto - se pasa la referencia
void funcion(Estadisticas& stats);
```

El paso por referencia (`&`) evita copias innecesarias y permite modificar los datos originales.

5. Constantes Simbólicas

```
#define BLOQUE_GRUESO "\u2588"
                       #define ESPACIO_VACIO " "
```

El uso de constantes hace el código más legible y fácil de mantener.

6. Comentarios Descriptivos

El código incluye comentarios que explican:

- El propósito de cada función
- El significado de variables importantes
- La lógica de algoritmos complejos

7. Diseño de Interfaz

Uso de caracteres Unicode y formato de cajas para mejorar la experiencia visual:

- ╔══╗ para marcos decorativos
- 🎮 🤖 🏆 emojis para mejor comunicación visual
- Mensajes claros y descriptivos

🚀 Posibles Mejoras y Extensiones

1. Persistencia de Datos

Implementación: Guardar estadísticas en un archivo `stats.txt`

```
// Al finalizar el programaofstream archivo("stats.txt");
archivo << stats.victorias << " " << stats.derrotas << " "
<< stats.empates << " " << stats.partidasJugadas;
archivo.close();
// Al iniciar el programa
ifstream archivo("stats.txt");
if(archivo.is_open()) {
archivo >> stats.victorias >> stats.derrotas
>> stats.empates >> stats.partidasJugadas;
archivo.close();
}
```

2. Algoritmo Minimax

Para una IA invencible, implementar el algoritmo Minimax con poda alfa-beta:

- Evalúa todos los posibles movimientos futuros
- Elige el movimiento óptimo
- Garantiza que la IA nunca pierda

3. Modo Multijugador en Red

Implementar sockets para jugar en red:

- Cliente-servidor usando TCP/IP
- Sincronización de tableros
- Sistema de turnos por red

4. Interfaz Gráfica (GUI)

Migrar a una interfaz gráfica usando bibliotecas como:

- **Qt:** Multiplataforma, completo
- **SFML:** Ligero, ideal para juegos
- **SDL:** Control bajo nivel

5. Sistema de Rankings

Implementar un sistema de puntuación Elo para clasificar jugadores.

6. Modos de Juego Adicionales

- Tablero 4x4 o 5x5
- Modo torneo (mejor de 3, 5, 7)
- Tiempo límite por movimiento
- Modo desafío con niveles predefinidos

7. Análisis de Partidas

Guardar el historial de movimientos y permitir:

- Reproducir partidas anteriores
- Análisis de errores
- Sugerencias de mejores movimientos

📝 Conclusión

Logros del Proyecto

Este proyecto implementa exitosamente un juego completo del Gato con las siguientes características destacadas:

- ✅ Sistema de IA con dos niveles de dificultad
- ✅ Validación robusta de entradas del usuario
- ✅ Interfaz de usuario atractiva en consola
- ✅ Sistema completo de estadísticas
- ✅ Código modular y bien organizado
- ✅ Múltiples modos de juego

Conceptos de Programación Aplicados

Concepto	Aplicación en el Proyecto
Estructuras	Jugador, Estadisticas
Arreglos Multidimensionales	Tablero 3x3
Funciones	Modularización completa
Paso por Referencia	Modificación de estructuras
Bucles Anidados	Recorrido del tablero
Condicionales	Validaciones y lógica de juego
Algoritmos	IA estratégica, verificación de ganador
Entrada/Salida	Interacción con usuario, formato visual

Valor Educativo

Este proyecto sirve como excelente ejemplo para aprender:

- Diseño de algoritmos de juegos
- Implementación de inteligencia artificial básica
- Validación y manejo de errores
- Organización de código en proyectos medianos
- Desarrollo de interfaces de usuario en consola

📚 Recursos para Seguir Aprendiendo

- **Algoritmo Minimax:** Para IA avanzada en juegos
- **Programación Orientada a Objetos:** Versión con clases
- **Estructuras de Datos:** Árboles de decisión para IA
- **Programación de Redes:** Versión multijugador
- **Interfaces Gráficas:** Migración a GUI

Universidad Veracruzana

Curso de Programación | 2024

Desarrollado por: Rodrigo Nuñez Garcia

Documentación generada para el proyecto académico
Juego del Gato - Versión 1.0.0